

July, 2026

An Energy-Centric Comparative Evaluation of Software-Based vs. Hardware-Accelerated Cryptography

on ESP32 in a Realistic
IoT Duty Cycle

Chapters 1–3 Presentation

Supervisor:
Dr. Keerthi Wijayasiriwardhane

Presented by :
Samoda Wijesooriya

Background

- IoT devices run on batteries, often for months or years without recharging
- Encryption is non-negotiable; it protects data privacy, integrity, and device identity
- Encryption protects data from eavesdropping, replay attacks, and side-channel attacks
- The wrong encryption choice can cut battery life in half.

Why we do this?

While having a dedicated hardware cryptographic engine built into modern microcontrollers like the ESP32, most IoT implementations still rely on software-based encryption. This raises a critical question: **Are we using our resources efficiently?**

Purpose of the Study

- To investigate why software encryption remains dominant even when hardware acceleration is available.
- To identify which approach, hardware or software, delivers the best balance of energy efficiency, speed, and resource utilization in real IoT duty-cycle conditions.

One Question. Three Measurable Objectives

"How does hardware-accelerated AES-128-GCM on the ESP32 compare with software-based ASCON-128 in terms of power consumption, memory usage, throughput, execution time, and per-byte energy efficiency when evaluated across a full IoT duty cycle (Sleep → Wake → Sense → Encrypt → Transmit → Sleep)?"

Objectives

- Compare total duty cycle energy and per-byte energy efficiency of ASCON-128 VS AES-128-GCM on the ESP32
- Measure the energy cost per transmitted byte across varying payload sizes and duty-cycle frequencies
- Identify break-even crossover conditions; the payload size and duty-cycle rate at which one algorithm becomes more efficient than the other

Research Literature Comparison

Author(s) & Year	What They Compared	Key Findings	Limitations/Gap
Jin & Becker, 2022	Crypto algorithm benchmarking on ESP32 (execution time, throughput)	Established foundational benchmarking framework for ESP32; hardware AES showed up to +257.8% throughput gain	Focused on speed only — no power/energy or duty-cycle analysis
Ridho Rabbani, 2025	Software ASCON-128 vs. software-emulated AES-128	Software ASCON cuts execution latency by 74% vs. software AES on the same ESP32	Benchmarked against software AES only; never tested against hardware-accelerated AES
Hung & Hsu, 2018	Software AES vs. hardware-accelerated AES (ARM Cortex-M3)	Hardware-stored AES matrices significantly reduce power draw and computation	Never compares against ASCON at all
Rafat et al., 2025	Hardware AES on ESP32	<3ms latency, 35–45% CPU usage with hardware acceleration	Same — hardware AES vs. software AES baseline, not ASCON

Research Literature Comparison

Author(s) & Year	What They Compared	Key Findings	Limitations/Gap
Tripathi et al., 2024	ASCON vs. software AES (Arduino-based 32-bit platforms)	ASCON often outperforms software AES in throughput/energy	No hardware-accelerated comparison; different platform family
Amirkhanova et al., 2026	End-to-end AES-GCM pipeline (ESP32, MQTT, Raspberry Pi)	Demonstrated practical encrypted IIoT pipeline	Narrow benchmark scope; did not quantify energy or CPU utilization
Nkemeni et al., 2023	Duty-cycle energy modeling (WSN water monitoring)	Identified startup/transient energy costs (~22μJ over 2.1ms) as significant	Validated mostly via simulation, not full crypto+duty-cycle physical testing
Dezfouli et al., 2018	Measurement instrumentation for IoT energy (EMPIOT)	Highlighted limitations of existing measurement tools (range, accuracy for current spikes)	Tooling-focused; not a crypto energy comparison itself

Research Literature Comparison

Author(s) & Year	What They Compared	Key Findings	Limitations/Gap
Syahmi Md Dzahir & Seng Chia, 2023	FreeRTOS on/off × deep sleep on/off (ESP32 MQTT monitoring)	Deep sleep improves battery runtime by 500%+	Doesn't isolate cryptographic overhead specifically
Beaulieu et al., 2013 / Al-Shmailawi et al., 2026	Metric standardization across LWC studies	Different studies use inconsistent code-size/energy definitions	No unified metric framework exists, making cross-study comparison unreliable

Summary of Research Gaps

- No reviewed study benchmarked software ASCON-128 against hardware-accelerated AES-128-GCM on the same platform
 - The majority of studies benchmark encryption as a continuous, isolated operation rather than embedding it inside a realistic duty cycle
 - Deep sleep wipes RAM, forcing full reboot/re-initialization of crypto parameters on every wake cycle
 - Most ESP32 crypto benchmarks (e.g., Jin & Becker, 2022; Amirkhanova et al., 2026) measure execution time/throughput but not power consumption or per-byte energy cost
- This thesis addresses these gaps by directly comparing AES-128-GCM (hardware) and ASCON-128 (software) on identical ESP32 hardware, across a full realistic duty cycle

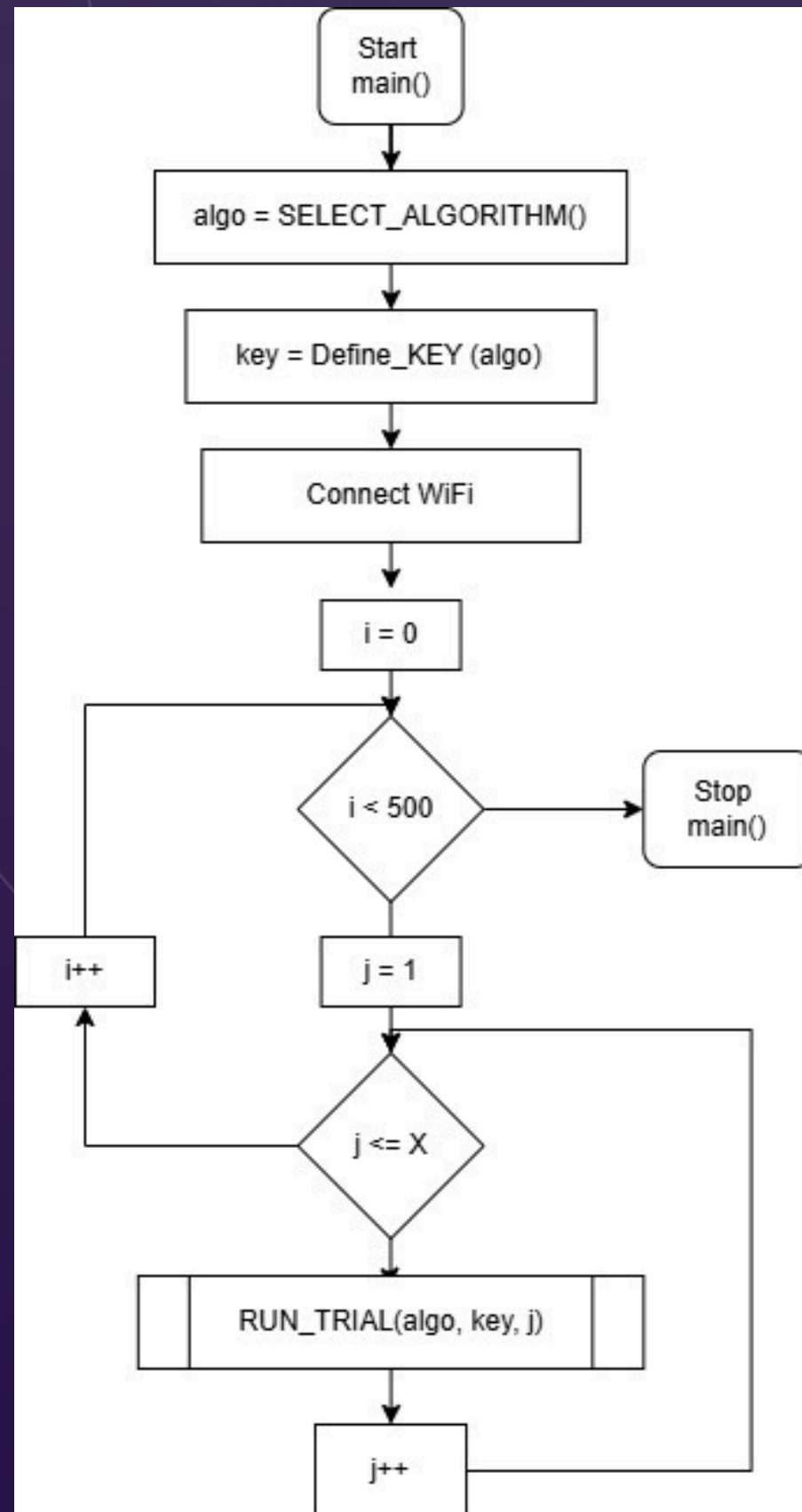
Methodology - Research Approach

- Quantitative Approach extends Jin & Becker (2022) benchmarking approach (execution time + throughput) by adding power consumption + realistic duty-cycle dimensions
- AES-128-GCM (hardware-accelerated):
 - Retained for compatibility with prior literature
 - GCM chosen over CBC for AEAD
 - To measure the ESP32's hardware engine performance at its best
- ASCON-128 (software):
 - NIST's 2023 lightweight standard; ESP32 has no hardware accelerator for it
 - Isolates pure software/algorithmic efficiency vs. hardware offloading

Methodology - Evaluation Metrics

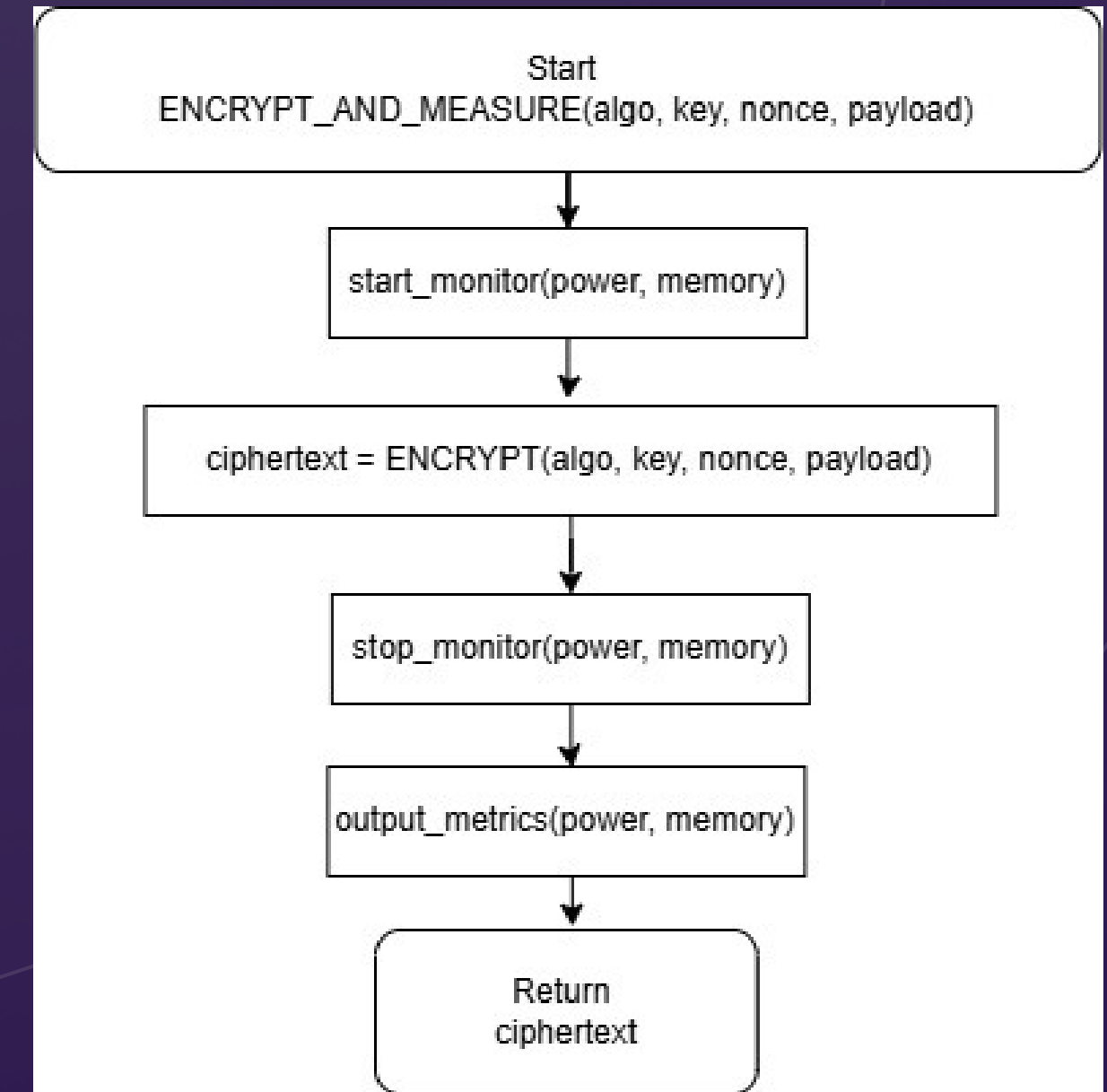
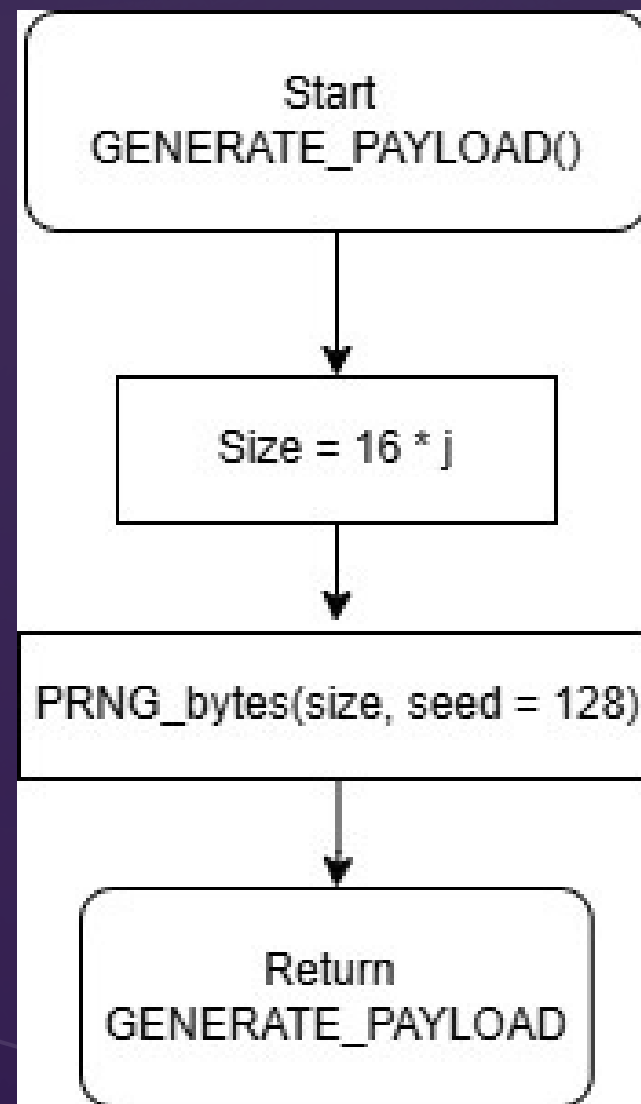
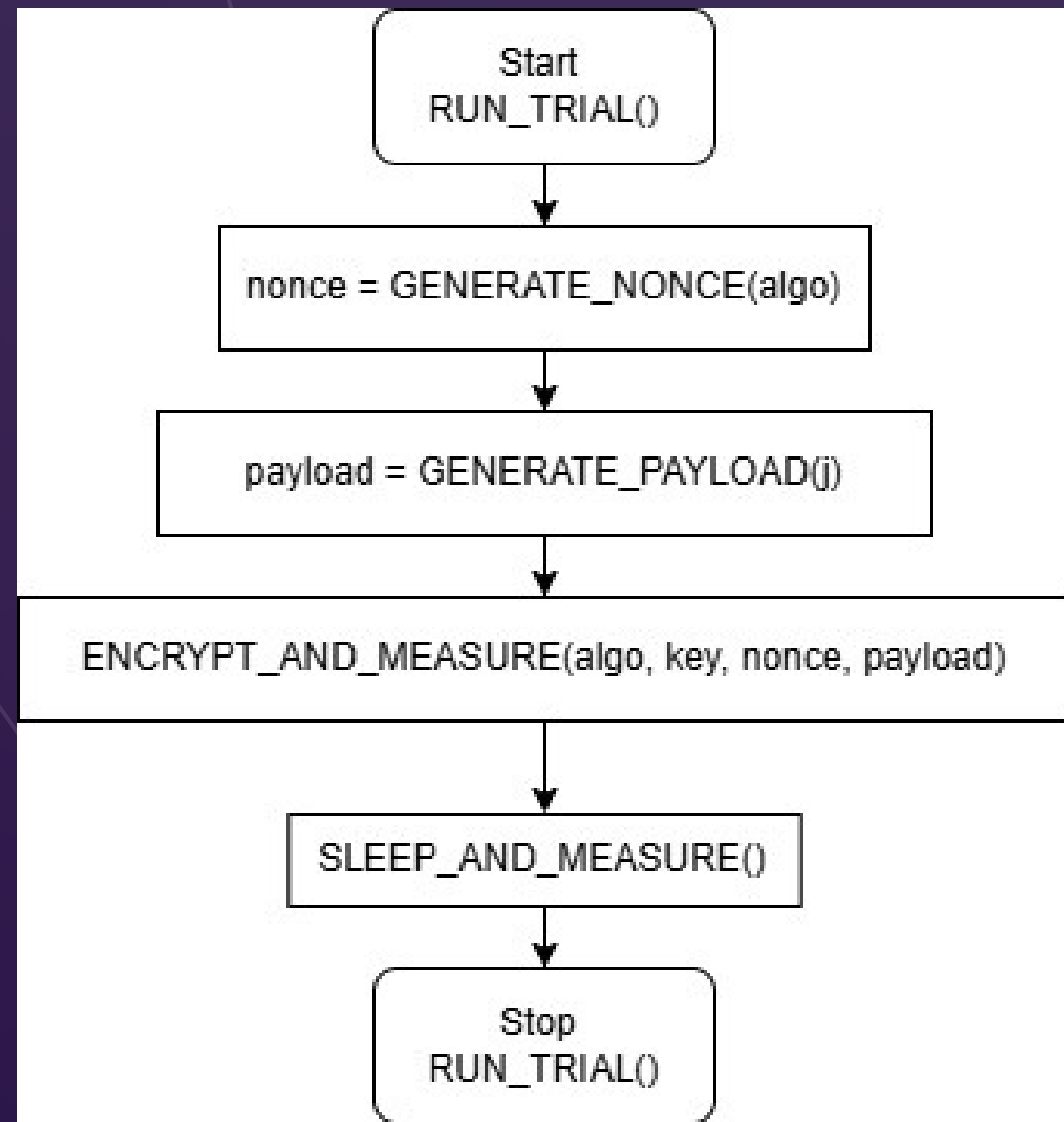
Metrics	Description	Measurement Method
Execution Time	The total duration required to complete a single cryptographic operation (encryption or decryption).	Measured using the built-in <code>micros()</code> function, capturing microsecond-resolution snapshots before and after each operation.
Throughput	The volume of data processed per unit of time, expressed in bytes per second (B/s).	Calculated using the formula: $\text{Throughput} = \text{Bytes in payload} / \text{mean execution time(s)}$
Power Consumption	The real-time electrical energy drawdown during active cryptographic processing.	Monitored via an INA219 current sensor, allowing for the tracking of milliampere-hours (mAh) or microjoules (μJ) per operation.
Memory Footprint	The impact of the cryptographic algorithm on the device's volatile memory (RAM).	Captured using the <code>ESP.getFreeHeap()</code> function to measure the difference in available heap memory before and after the process.

Methodology - Experimental Design

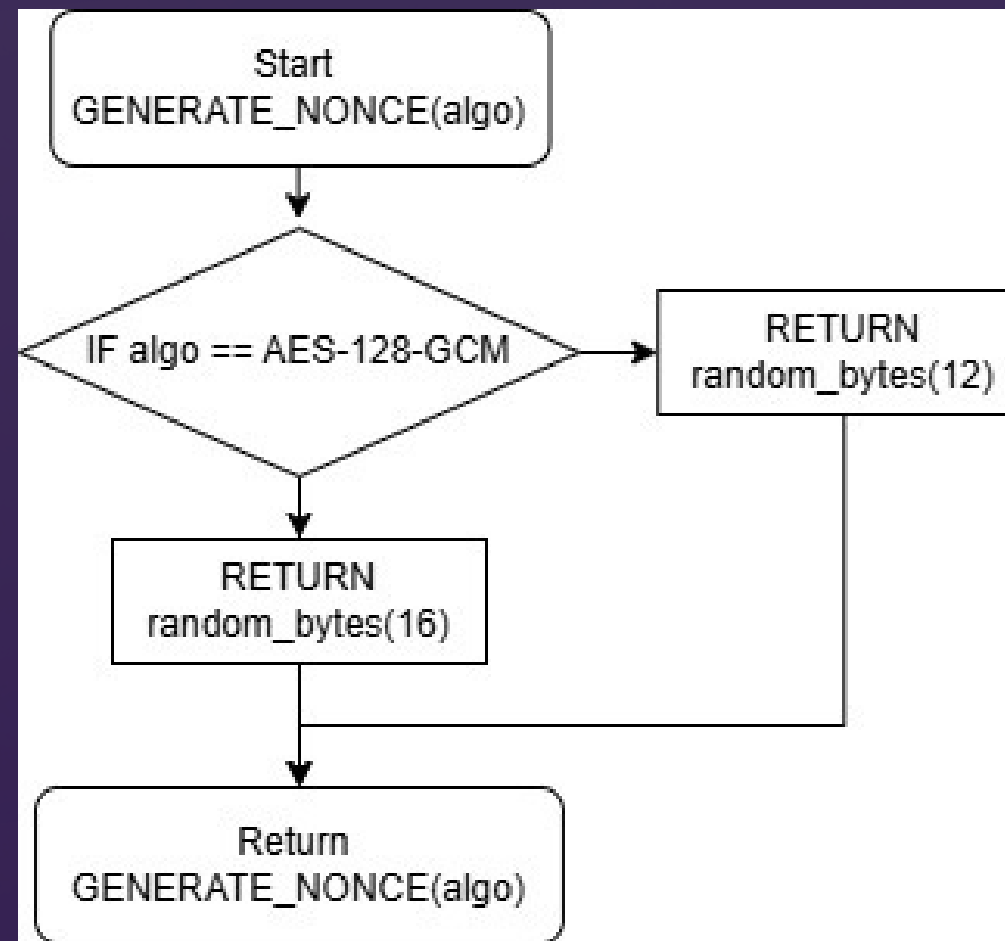
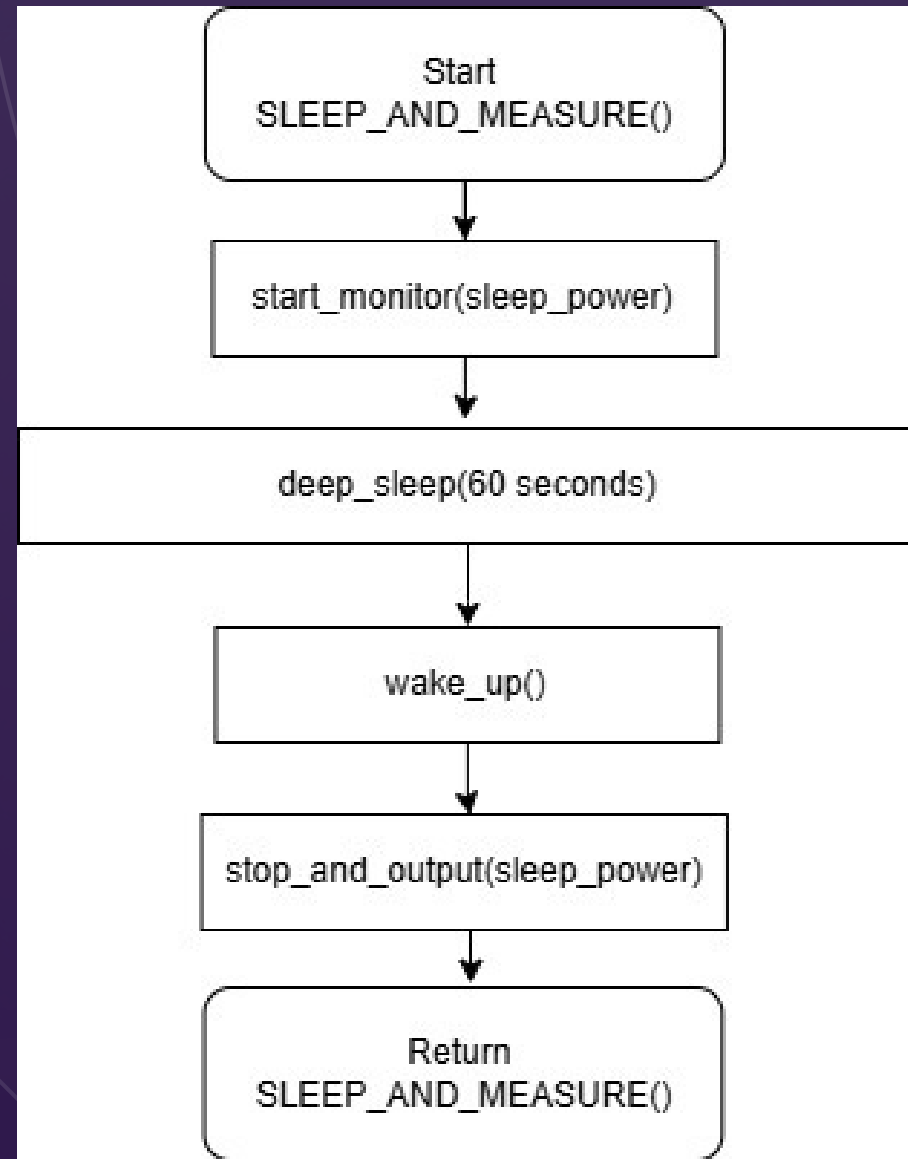


- Outer loop: 500 iterations per data point (statistical stability)
- Inner loop: 100 payload sizes, $\text{Size} = 16 \times j \rightarrow$ payloads from 16 to 1,600 bytes
- Payloads generated via PRNG seeded at 128 for reproducibility
- All the metrics are printed to the console and transmitted to Node-Red using WiFi

Methodology - Experimental Design

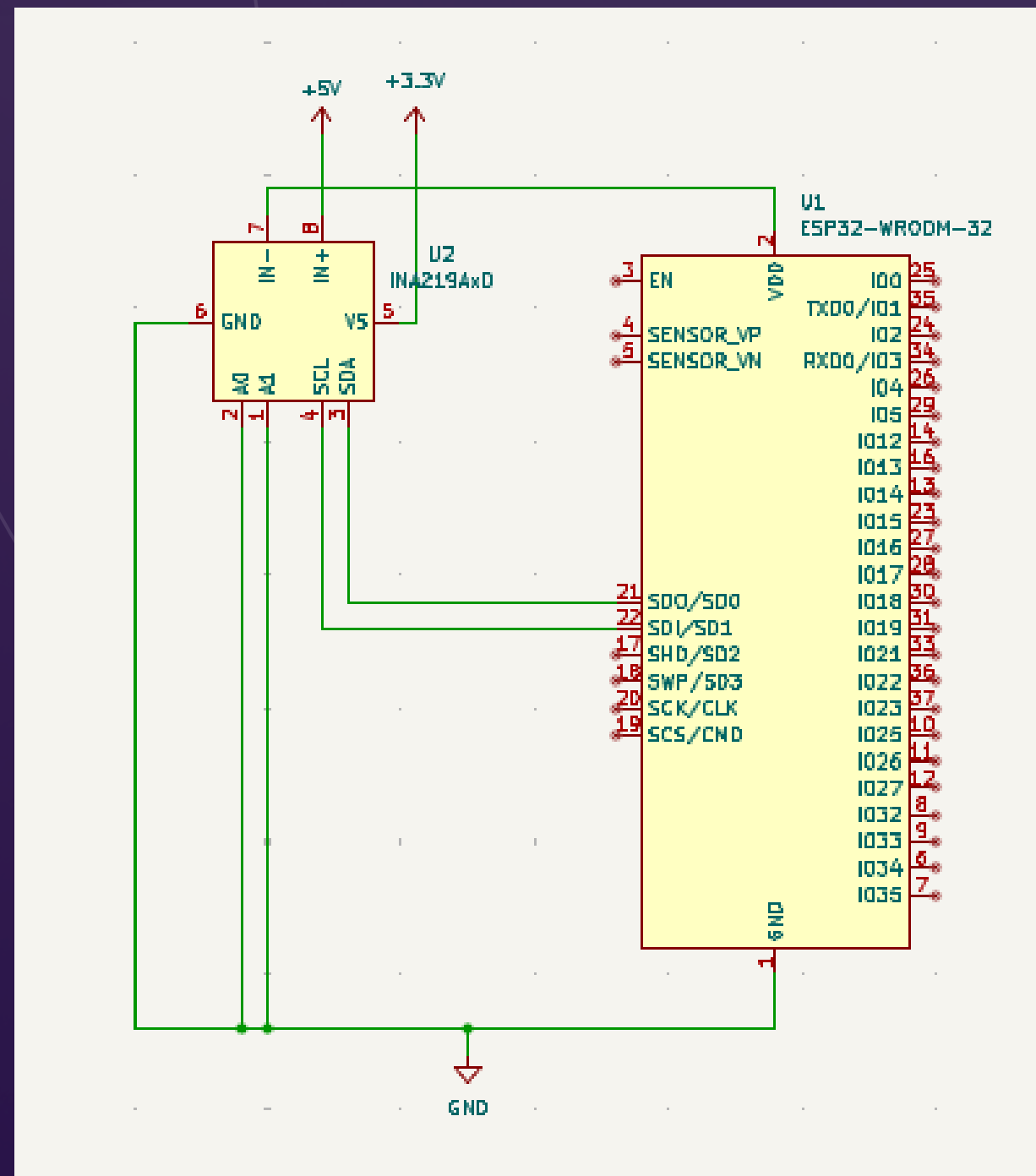


Methodology - Experimental Design

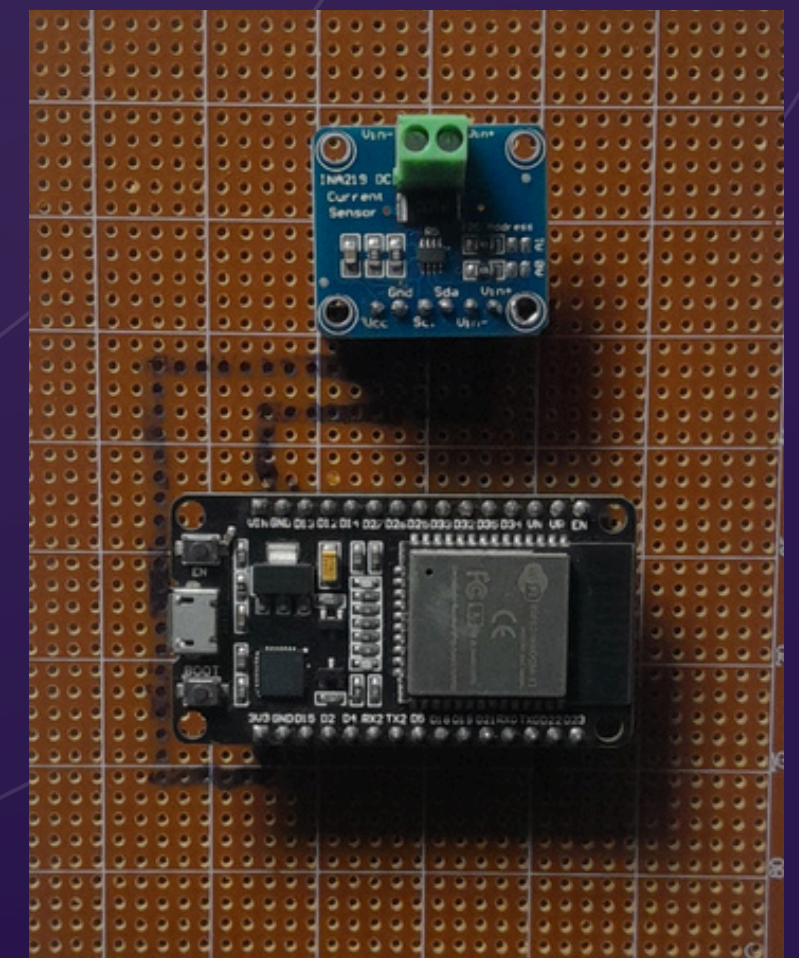


- 60-second deep sleep between each active session (based on Syahmi Md Dzahir & Seng Chia, 2023 methodology)

Methodology - Test-bed Hardware



- ESP32-WROOM-32E: dual-core Xtensa LX6 @ up to 240MHz, ~400KB SRAM, up to 16MB external flash, built-in AES/SHA/RSA/RNG hardware accelerators, deep sleep down to 5–10 μ A.
- INA219 current/power sensor: high-side I²C sensor, measures current & power directly, connected via GPIO 21 (SDA)/22 (SCL), I²C address 0x40.
- INA219 placed in series between 5V supply and ESP32 load.
- Soldered dot-board assembly (not breadboard) to eliminate contact-resistance noise.

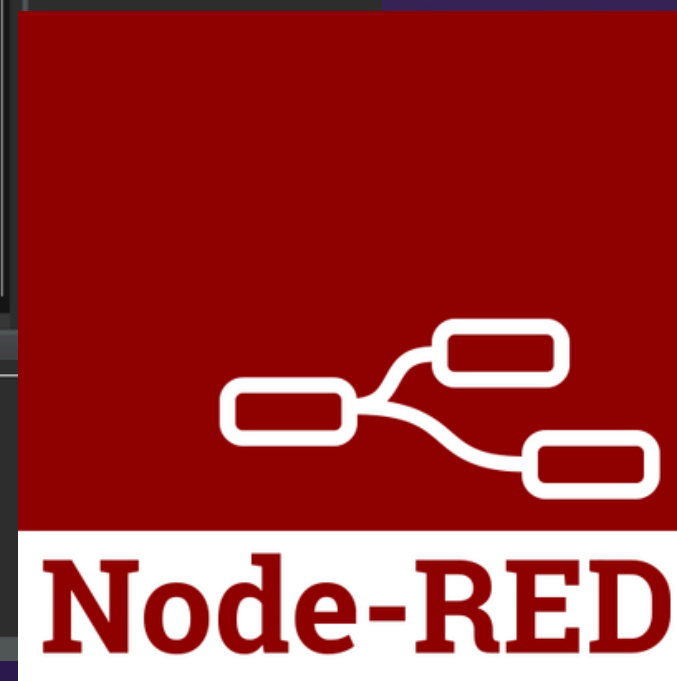
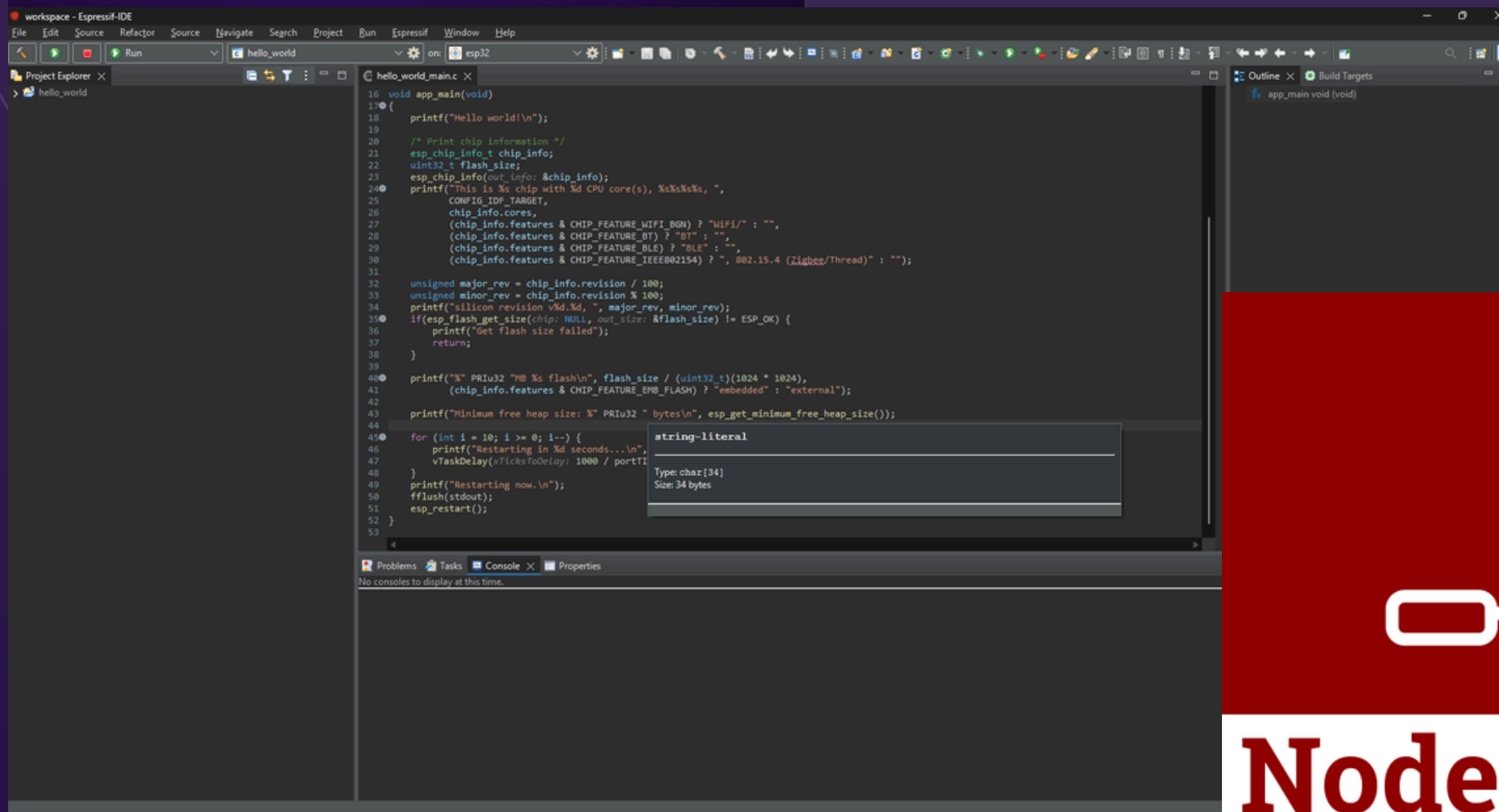


Methodology - Software Env



- ESP-IDF framework, ANSI C, gcc
- mbedTLS 2.16.2 (hardware AES interface — must explicitly invoke hardware or it silently falls back to software)
- ESP-IDF MQTT library 3.1.1 + Mosquitto broker 1.6.7 for transmitting data

- Node-RED acts as the cloud server in the WSN network



Methodology - Measurements Formulas

$$\text{Throughput(B/s)} = \frac{(16 \times j) \text{ Bytes}}{\text{Mean Execution Time (Seconds)}}$$

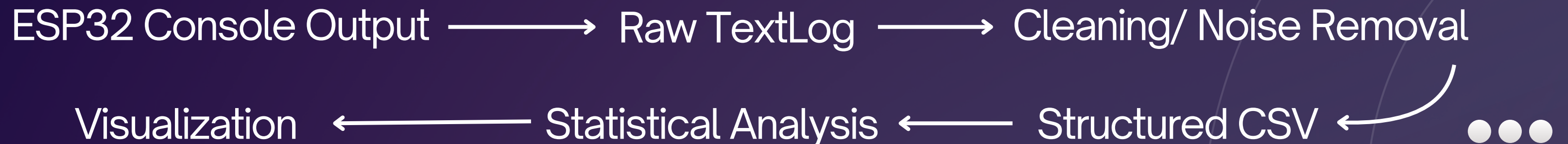
- Measures how fast each algorithm can process data, in bytes per second (B/s).
- Memory: gc.collect() → snapshot before/after → difference = RAM used

$$P = V \times I$$

- Applied here to get instantaneous power draw during each phase (encryption, sleep, transmit)

Data Collection & Extraction Pipeline

- Each cryptographic operation generates two core metrics on-device: execution time and memory usage
- Metrics are output to the ESP-IDF console, where raw data is captured and saved to a text file (transmitted live via MQTT to the Node-RED dashboard)
- Saved output goes through a data extraction and cleaning process
- Cleaned logs are restructured into CSV files for easier statistical handling
- Cleaned data is visualized through tables and graphs to identify performance trends across encryption modes and payload sizes



Statistical Methods & Tools

- Median is used to represent typical performance (Reduces the skewing effect of irregular spikes in execution time)
- Mean execution time & throughput are also calculated for large-scale datasets (1000+ iterations)
- Standard deviation & 95% confidence intervals computed to quantify variability between iterations
- Python's SciPy library: used for p-value evaluation and cleaning raw serial communication data (per Mitchel et al., 2025)



Expected Outcomes

- Direct empirical AES-128-GCM (HW) vs ASCON-128 (SW) comparison: never done before on the same platform
- Per-byte energy profile across 16–1,600 byte payloads, including sleep/init overhead
- Identified break-even payload size: a concrete decision threshold, not a general claim
- A reusable physical benchmarking methodology (ESP32 + INA219) for future IoT crypto studies
- Practical guidance on when to use hardware vs software crypto in real IoT deployments



Thank You

Q & A